



Capítulo 11 - Regras de segurança para os desvios

Antes de movimentar um desvio, o programa precisa saber se o comando pode ser executado.

Introdução	No capítulo anterior, reservamos recursos para uma rota. Agora vamos aprofundar as regras que impedem comandos de desvios em condições inadequadas.
Objetivo	Organizar permissões, bloqueios e recuperação para o comando de um desvio, diferenciando condições conhecidas, ocupação, reserva e emergência.
Meta de aprendizagem	Ao final, você deverá conseguir verificar um pedido antes de enviá-lo, informar o motivo de uma recusa e exigir uma nova solicitação depois de retirar um bloqueio.

1. Por que bloquear um comando?

Movimentar as agulhas enquanto uma composição passa pelo desvio pode provocar um movimento inadequado ou descarrilamento. Um comando também pode comprometer uma rota já preparada para outro movimento.

Por isso, a solicitação do operador não deve acionar diretamente o servo. Primeiro, o programa verifica se as condições permitem a mudança.

“Quero D6 em AC” é um pedido. “A região está disponível e o comando pode ser enviado” é uma decisão. Manter essas duas etapas separadas facilita aplicar as regras a todos os pedidos.

2. Condições conhecidas e condições desconhecidas

Uma leitura que não indica acionamento não basta para comprovar que a região do desvio está livre. É necessário considerar os limites da detecção e a passagem de toda a composição.

Ao iniciar a ESP32 ou perder a confiança nas informações, o programa deve tratar a disponibilidade como desconhecida e recusar uma nova movimentação até a conferência.

O estado anterior guardado na memória não garante que nada mudou durante uma falha, desligamento ou intervenção manual.

Na Ferrovia EFMN 74, não utilizamos sensores Hall nas agulhas dos desvios. Os sensores de passagem devem ser interpretados pela lógica dos trechos, sem tratá-los como confirmação mecânica do servo.



3. Regras para aceitar uma mudança

Condição	Regra do exercício
Informações válidas	Sem informação confiável, recusar o pedido.
Região do desvio livre	Com ocupação, recusar o movimento.
Sem reserva por rota	Um pedido individual não pode alterar um recurso reservado.
Sem emergência	Uma emergência impede novos comandos.
Sem bloqueio manual	Uma intervenção exige bloquear o comando.
Sem movimento em andamento	Um novo pedido não deve substituir o movimento atual.

Todas essas condições precisam ser atendidas. O código deve informar o motivo da recusa para ajudar o operador a compreender a situação.

A ocupação relevante é a região que pode ser afetada pelo movimento das agulhas, e não apenas o ponto exato onde um sensor foi instalado.

4. Reserva por rota e pedido individual

Uma rota pode reservar um desvio para seu percurso. Enquanto essa reserva existir, um pedido individual não deve alterar a posição estabelecida.

Na integração futura, a função de preparação da rota precisará identificar a própria reserva e seus recursos. Uma reserva da rota que está sendo preparada não deve ser confundida com permissão para qualquer comando externo.

O exercício deste capítulo utiliza um bloqueio simples chamado `routeReserved`. Ele representa uma reserva já existente e impede todos os pedidos individuais.

Retirar uma reserva exige verificar a disponibilidade do percurso. A passagem da locomotiva por um sensor de saída não prova que todos os vagões deixaram a região do desvio.

5. Comando de posição e confirmação física

O programa pode guardar a última posição comandada, mas isso não confirma que as agulhas chegaram a essa posição. Pode haver haste solta, curso insuficiente, resistência mecânica ou falha de alimentação.

Também não devemos considerar que um servo terminou apenas porque transcorreu um intervalo. No exercício, utilizaremos um segundo como espera didática entre comandos, sem confirmação física.



Em uma operação real, a confiança na posição precisa ser definida no projeto: conferência manual, mecanismo de detecção adequado ou outra solução validada. O código deste capítulo não acrescenta sensores aos desvios.

6. Emergência e recuperação

A emergência bloqueia novas solicitações. Não deve provocar um retorno automático do desvio para DI, pois essa mudança pode ocorrer enquanto uma composição está passando.

Retirar uma emergência não executa o pedido que foi recusado anteriormente. O operador deve conferir as condições e fazer uma nova solicitação.

Se a emergência surgir depois de um comando já ter sido enviado ao servo, o programa pode impedir comandos seguintes, mas não pode apagar fisicamente o movimento iniciado. O tratamento de energia e tração depende do projeto elétrico.

O bloqueio de software não substitui o comando de emergência da gaveta de controle principal. Este exercício não comanda seus relés nem a alimentação dos servos da gaveta GA3.

7. Simulação de um desvio

Utilizaremos D6 como exemplo de desvio da gaveta GA3. O teste será feito somente pelo Monitor Serial, sem biblioteca de servo, sem GPIO de controle e sem movimentação física.

Comando	Ação simulada
V / I	Informações válidas / inválidas.
L / O	Região livre / ocupada.
T / U	Reservar por rota / retirar a reserva após conferência.
B / C	Ativar / retirar bloqueio manual.
E / R	Ativar / retirar emergência.
1 / 2	Solicitar D6 em DI / AC.

U representa uma conferência manual e somente é aceito quando as informações são válidas, a região está livre, não existe emergência e não há comando em andamento.

8. Programa completo

Crie `intermediate_11_turnout_safety`. Copie todos os blocos abaixo, na ordem, para o mesmo programa.



```
bool informationValid = false;
bool areaClear = false;
bool routeReserved = false;
bool manualBlocked = true;
bool emergencyActive = false;
bool movementPending = false;
unsigned long movementStarted = 0;
const unsigned long movementWait = 1000;

bool commandAllowed() {
    if (emergencyActive) {
        Serial.println("Rejected: emergency active");
    } else if (manualBlocked) {
        Serial.println("Rejected: manual block");
    } else if (!informationValid) {
        Serial.println("Rejected: information unknown");
    } else if (!areaClear) {
        Serial.println("Rejected: turnout area occupied");
    } else if (routeReserved) {
        Serial.println("Rejected: resource reserved");
    } else if (movementPending) {
        Serial.println("Rejected: previous command pending");
    } else {
        return true;
    }
    return false;
}

void requestTurnout(bool diverging) {
    if (!commandAllowed()) return;
    Serial.println(diverging ? "Simulated D6 command: AC"
                             : "Simulated D6 command: DI");
    movementPending = true;
    movementStarted = millis();
}

void updateMovementWait() {
    if (movementPending &&
        millis() - movementStarted >= movementWait) {
        movementPending = false;
        Serial.println("Wait completed - physical position NOT confirmed");
    }
}

void releaseRoute() {
    if (!informationValid || !areaClear || emergencyActive
        || movementPending) {
        Serial.println("Reservation retained - check conditions");
        return;
    }
    routeReserved = false;
    Serial.println("Reservation released by operator");
}
```



```
void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("EFMN 74 - D6 command safety simulation");
  Serial.println("Startup: manual block and unknown conditions");
  Serial.println("Commands: V I L O T U B C E R 1 2");
}

void loop() {
  updateMovementWait();
  if (Serial.available() > 0) {
    char command = Serial.read();
    switch (command) {
      case 'V': informationValid = true; break;
      case 'I': informationValid = false; break;
      case 'L': areaClear = true; break;
      case 'O': areaClear = false; break;
      case 'T': routeReserved = true; break;
      case 'U': releaseRoute(); break;
      case 'B': manualBlocked = true; break;
      case 'C': manualBlocked = false; break;
      case 'E': emergencyActive = true; break;
      case 'R': emergencyActive = false; break;
      case '1': requestTurnout(false); break;
      case '2': requestTurnout(true); break;
      default:
        if (command != '\n' && command != '\r') {
          Serial.println("Invalid command");
        }
        break;
    }
  }
}
```

9. Entendendo as recusas

commandAllowed() verifica as condições em uma ordem definida. Quando houver mais de um impedimento, informa o primeiro encontrado. Depois que ele for retirado, uma nova solicitação poderá revelar outro motivo de bloqueio.

requestTurnout() só apresenta o comando se a verificação retornar true. Pedidos recusados não ficam em uma fila para execução futura.

movementPending impede outro pedido durante a espera. Ao terminar, a mensagem deixa explícito que a posição física não foi confirmada.

O acompanhamento do tempo continua em loop(). Assim, o programa pode receber o bloqueio ou a emergência durante a espera.

Depois de uma emergência ou perda de informação durante o movimento, o operador deve revisar a posição física antes de voltar a operar, mesmo que a espera já tenha terminado.

Exercício prático

Meta: confirmar que cada condição impede um pedido individual e que a recuperação exige nova solicitação.



1. Conecte a ESP32 de bancada por USB, selecione placa e porta, compile e envie.
2. Abra o Monitor Serial em 115200 baud e pressione EN ou RESET.
3. Envie 1. Confira a recusa pelo bloqueio manual.
4. Envie C e depois 1. Confira a recusa por informações desconhecidas.
5. Envie V e depois 1. Confira a recusa por região considerada ocupada ou indisponível.
6. Envie L e depois 1. Confira o comando simulado de DI.
7. Antes de um segundo, envie 2. Confira a recusa pelo comando pendente.
8. Aguarde a mensagem de fim da espera e envie 2 novamente. Confira AC.
9. Aguarde o término, envie T e depois 1. Confira a recusa por reserva.
10. Envie O e tente U. A reserva deverá ser mantida.
11. Simule a conferência da saída da composição inteira, envie L e depois U.
12. Envie E e depois 1. Confira a recusa por emergência.
13. Envie R. Nenhum comando de desvio deverá ser executado automaticamente.
14. Confira as condições e envie 1 novamente.

Exemplos de mensagens:

```
Rejected: manual block  
Rejected: information unknown  
Rejected: turnout area occupied  
Simulated D6 command: DI  
Rejected: previous command pending  
Wait completed - physical position NOT confirmed
```

Para testar o bloqueio por comando pendente com facilidade, envie 12 em uma única transmissão. O programa processará o pedido de DI e depois recusará o pedido seguinte durante a espera. Esse envio combinado serve apenas para o teste.

10. Conferências para a montagem real

- Identifique o desvio e a gaveta responsável: no exemplo, D6 da gaveta GA3.
- Confira as posições DI e AC com o TTD-01 e registre os ângulos ajustados.
- Verifique a fixação do servo, a haste e o encosto suave das agulhas.
- Confira a alimentação, o GND comum e a distribuição dos sinais.
- Defina quais regiões precisam estar livres para permitir a mudança.
- Identifique quais rotas reservam o desvio e quem pode retirar cada reserva.
- Defina como a operação recuperará a confiança na posição após uma falha.



Na integração, a função de comando deve consultar essas permissões antes de enviar a posição ao servo. Todas as origens de pedido, inclusive comandos automáticos e manuais, precisam respeitar as regras pertinentes.

Manter um desvio na posição de repouso não significa que ele possa retornar a essa posição a qualquer momento. Uma mudança deve aguardar disponibilidade e condições verificadas.

Em uma intervenção na gaveta GA3, bloqueie os comandos e desligue a alimentação pertinente antes de alterar ligações ou mecanismos. A indicação de bloqueio no programa não garante que o circuito esteja desenergizado.

Resumo do capítulo

- Todo pedido deve ser verificado antes do comando.
- Condição desconhecida não equivale a região livre.
- Ocupação, reserva, emergência e intervenção podem impedir a mudança.
- Um comando em andamento não deve ser substituído por outro pedido.
- Tempo de espera não confirma a posição física das agulhas.
- A emergência não deve provocar retorno automático para DI.
- Pedidos recusados não são executados automaticamente após a recuperação.
- A lógica de software precisa trabalhar em conjunto com a montagem e as proteções elétricas.

Próximo capítulo: Integração entre sensores, desvios e tração.